

PA3-穿越时空的旅程：异常控制流

世界诞生的故事-第三章

冯诺依曼计算机果然功力深厚,竟然能向冷冰冰的门电路赋予新的生命。但为了应对各种突发情况,先驱对计算机进行了改进。

在进行本 PA 前,请在工程目录下执行以下命令进行分支整理,否则将影响你的成绩:

```
git commit --allow-empty -am "before starting pa3"
git checkout master
git merge pa2
git checkout -b pa3
```

操作系统 - 更方便的运行时环境

我们在 PA2 中已经实现了一个冯诺依曼计算机系统,并且已经在 AM 上把打字游戏运行起来了。有了 IOE,几乎能把各种小游戏移植到 AM 上来运行了。但说起运行仙剑奇侠传,我们目前暂时还无能为力。这主要是因为,仙剑奇侠传算得上是一个较为复杂的游戏了,它会使用文件来管理游戏相关的数据。一提到文件,就已经超出了 AM 的能力范围了,因为 AM 只是计算机的一种抽象模型,是用来描述计算机如何构成的,作为运行时环境对程序的支撑能力也很有限,显然文件的概念并不属于 AM。

事实上,我们每天都使用的文件,其实是操作系统提供的一种服务。仔细想想,我们使用的绝大部分程序,都是在操作系统上运行的,这是因为操作系统的层次比 ISA 和 AM 都要高,自然能提供更丰富的抽象和更方便的运行时环境。如果要让开发者在 AM 上进行开发,估计各种游戏都早已陷入无尽跳票的死循环中了。

因此,为了运行规模更大的程序,我们需要操作系统的支持。噢,可别被操作系统这个庞然大物吓到了,我们只需要一个支持文件操作的操作系统,就可以支撑仙剑奇侠传的运行了。感觉还是比较复杂啊,我们还是先做一件最简单的事情:先实现一个足够简单的操作系统,来支撑 dummy 程序的运行。

RTFSC (4)

框架代码中已经为大家准备好了 Nanos-lite 的代码。Nanos-lite 是操作系统 Nanos 的裁剪版,是一个为 PA 量身订造的操作系统。换句话说,我们现在就要在 NEMU 上运行一个操作系统了(尽管这是一个比较简陋的操作系统),同时也将带领你根据课堂上的知识剖析一个简单操作系统的组成。这不仅是作为对这些抽象知识的很好的复(预)习,同时也是为以后的操作系统实验打下坚实的基础,而对 PA 来说最重要的是,体会操作系统对运行程序的意义所在。

Nanos-lite 已经包含了后续 PA 用到的所有模块,由于 NEMU 的功能是逐渐添加的,Nanos-lite 也要配合这个过程,你会通过 nanos-lite/src/main.c 中的一些与实验进度相关的宏来控制 Nanos-lite 的功能。随着实验进度的推进,我们会逐渐讲解所有的模块,Nanos-lite 做的工作也会越来越多。因此在阅读 Nanos-lite 的代码时,你只需要关心和当前进度相关的模块就可以了,不要纠缠于和当前进度无关的代码。

```
nanos-lite
├── include
│   ├── common.h
│   ├── debug.h
│   ├── fs.h
│   ├── memory.h
│   └── proc.h
├── Makefile
└── src
    ├── device.c # 设备抽象
    ├── fs.c     # 文件系统
    ├── initrd.S # ramdisk设备
    ├── irq.c   # 中断异常处理
    ├── loader.c # 加载器
    ├── main.c
    ├── mm.c    # 存储管理
    ├── proc.c  # 进程调度
    ├── ramdisk.c # ramdisk驱动程序
    └── syscall.c # 系统调用处理
```

需要提醒的是,Nanos-lite 是运行在 AM 之上的,AM 的 API 在 Nanos-lite 中都是可用的. 因此我们会有以下说法:

Nanos-lite 是 NEMU 的客户程序, 它运行在 AM 之上.
仙剑是 Nanos-lite 的用户程序, 它运行在 Nanos-lite 之上.

另外, 虽然不会引起明显的误解, 但在引入 Nanos-lite 之后, 我们还是会某些地方使用"用户进程"的概念, 而不是"用户程序". 如果你现在不能理解什么是进程, 你只需要把进程作为"正在运行的程序"来理解就可以了. 还感觉不出这两者的区别? 举一个简单的例子吧, 如果你打开了记事本 3 次, 计算机上就会有 3 个记事本进程在运行, 但磁盘中的记事本程序只有一个. 进程是操作系统中一个重要的概念, 有关进程的详细知识会在操作系统课上进行介绍.

一开始, 在 nanos-lite/src/main.c 中所有与实验进度相关的宏都没有定义, 此时 Nanos-lite 的功能十分简单. 我们来简单梳理一下 Nanos-lite 目前的行为:

- ❖ 通过 Log() 输出 hello 信息和编译时间. 需要说明的是, Nanos-lite 中定义的 Log() 宏并不是 NEMU 中定义的 Log() 宏. Nanos-lite 和 NEMU 是两个独立的项目, 它们的代码不会相互影响, 你在阅读代码的时候需要注意这一点. 在 Nanos-lite 中, Log() 宏通过 klib 中的 printk() 输出, 最终会调用 TRM 的 _putc().
- ❖ 初始化 ramdisk. 在一个完整的模拟器中, 程序应该存放在磁盘中. 但目前我们并没有实现磁盘的模拟, 因此先把 Nanos-lite 中的一段内存作为磁盘来使用. 这样的磁盘有一个专门的名字, 叫 ramdisk.
- ❖ 调用 init_device() 对设备进行一些初始化操作. 目前 init_device() 会直接调用 _ioe_init().
- ❖ 调用 loader() 函数加载用户程序, 函数会返回用户程序的入口地址. 其中 loader() 函数并未实现, 我们会在下文进行说明.
- ❖ 跳转到用户程序的入口执行.

加载操作系统的第一个用户程序

loader 是一个用于加载程序的模块. 我们知道程序中包括代码和数据, 它们都是存储在可执行文件中. 加载的过程就是把可执行文件中的代码和数据放置在正确的内存位置, 然后跳转到程序入口, 程序就开始执行了. 更具体的, 为了实现 loader() 函数, 我们需要解决以下问题:

- ❖ 可执行文件在哪里?
- ❖ 代码和数据在可执行文件的哪个位置?
- ❖ 代码和数据有多少?
- ❖ "正确的内存位置"在哪里?

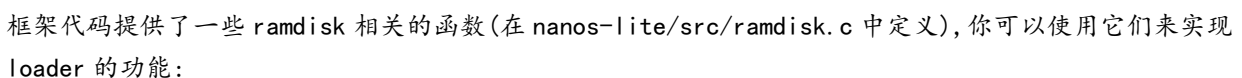
为了回答第一个问题, 我们还要先说明一下用户程序是从哪里来的. 由于用户程序运行在操作系统之上, 不能与 AM 所提供的运行时环境相适配了, 因此我们不能把编译到 AM 上的程序放到操作系统上运行. 为此, 我们准备了一个新的子项目 Navy-apps, 专门用于编译出操作系统的用户程序.

```
navy-apps
├── apps                # 用户程序
│   ├── init
│   ├── litenes
│   ├── lua
│   ├── nterm
│   ├── nwm
│   └── pal            # 仙剑奇侠传
├── fsimg              # 根文件系统
├── libs               # 库
│   ├── libc          # Newlib C库
│   ├── libfont
│   ├── libndl
│   └── libos         # 系统调用的用户层封装
├── Makefile
├── Makefile.app
├── Makefile.check
├── Makefile.compile
├── Makefile.lib
├── README.md
└── tests             # 一些测试
```

其中, navy-apps/libs/libc 中是一个名为 Newlib 的项目, 它是一个专门为嵌入式系统提供的 C 库, 库中的函数对运行时环境的要求极低. 这对 Nanos-lite 来说是非常友好的, 我们不需要为了配合 C 库而在 Nanos-lite 中实现额外的功能. 用户程序的入口位于 navy-apps/libs/libc/start.c 中的 _start() 函数, 它会调用用户程序

我们要在 Nanos-lite 上运行的第一个用户程序是 `navy-apps/tests/dummy/dummy.c`。首先我们让 Navy-apps 项目上的程序默认编译到 x86 中：

所以,目前的 loader 只需要做一件事情:将 ramdisk 中从 0 开始的所有内容放置在 0x4000000,并把这个地址作为程序的入口返回即可.我们把这个简化了的 loader 称为 raw program loader.我们通过内存布局来理解 loader 目前需要做的事情:



真实操作系统中的 loader 远比我们目前在 Nanos-lite 中实现的 loader 要复杂。事实上, Nanos-lite 的 loader 设计其实也向我们展现出了程序的最为原始的状态:一个凝结着人类智慧设计的精妙算法,承载着人类劳动收集的宝贵数据的... 比特币!加载程序其实就是把这一无比珍贵的比特币串放置在正确的位置,但这看似平凡无比的比特币串当中又蕴含着"存储程序"的划时代思想:当操作系统将控制权交给它的时候,计算机以把它解释成指令并逐条执行,却让这一比特币串真正发挥出它足以改变世界的潜能。

实现 loader

你需要在 Nanos-lite 中实现 loader 的功能,来把用户程序加载到正确的内存位置,然后执行用户程序.需要注意的是,每当 ramdisk 中的内容需要更新时,你都需要在 nanos-lite/目录下手动执行

```
make update
```

来更新 Nanos-lite 中的 ramdisk 内容,然后再通过

```
make run
```

来在 NEMU 上运行带有最新版 ramdisk 的 Nanos-lite.

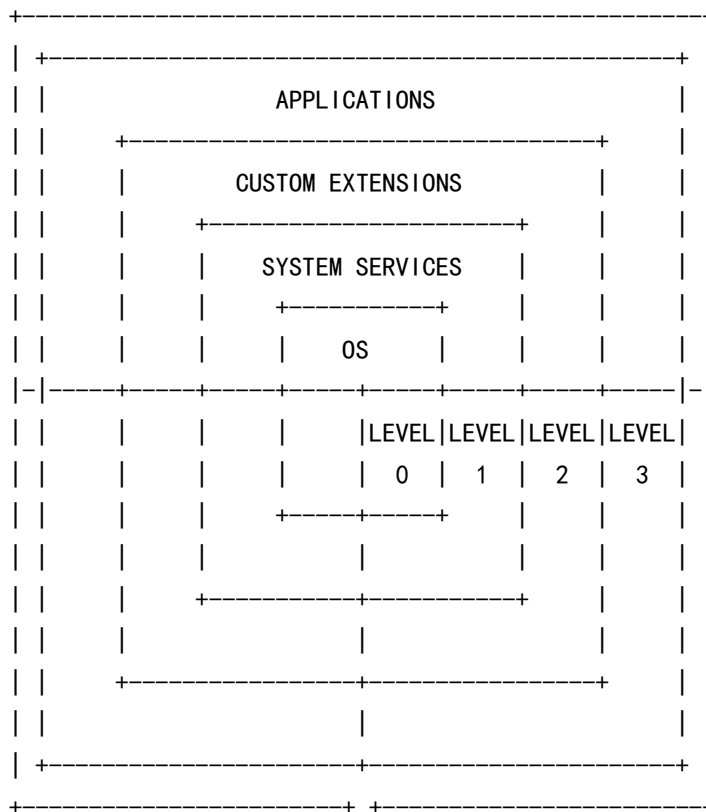
实现正确后,你会看到 dummy 程序执行了一条未实现的 int 指令,这说明 loader 已经成功加载 dummy,并且成功地跳转到 dummy 中执行了.未实现的 int 指令我们会在接下来的内容中进行说明.

等级森严的制度

我们在 dummy 程序中碰到了一条看似奇怪的 int 指令.为了解释它,我们还需要了解它背后折射出来的计算机和谐社会的故事.

为了构建计算机和谐社会,i386 强化了保护模式(protected mode)和特权级(privilege level)的概念:简单地说,只有高特权级的进程才能去执行一些系统级别的操作,如果一个特权级低的进程尝试执行它没有权限执行的操作,CPU 将会抛出一个异常.一般来说,最适合担任系统管理员的角色就是操作系统了,它拥有最高的特权级,可以执行所有操作;而除非经过允许,运行在操作系统上的用户进程一般都处于最低的特权级,如果它试图破坏社会的和谐,它将会被判"死刑".

在 i386 中,存在 0,1,2,3 四个特权级,0 特权级最高,3 特权级最低.特权级 n 所能访问的资源,在特权级 0~n 也能访问.不同特权级之间的关系就形成了一个环:内环可以访问外环的资源,但外环不能进入内环的区域,因此也有"ring n"的说法来描述一个进程所在的特权级.



虽然 80386 提供了 4 个特权级,但大多数通用的操作系统只会使用 0 级和 3 级:操作系统处在 ring 0,一般的程序处在 ring 3,这就已经起到保护的作用了.那 CPU 是怎么判断一个进程是否执行了无权限操作呢?在这之前,我们还要简单地了解一下 i386 中引入的与特权级相关的概念:

- ❖ DPL (Descriptor Privilege Level) 属性描述了一段数据所在的特权级
- ❖ RPL (Requestor's Privilege Level) 属性描述了请求者所在的特权级
- ❖ CPL (Current Privilege Level) 属性描述了当前进程的特权级,

一次数据的访问操作是合法的,当且仅当

```
data.DPL >= requestor.RPL      #<1>
```

```
data.DPL >= current_process.CPL  #<2>
```

两式同时成立,注意这里的>=是数值上的(numerically greater).<1>式表示请求者有权限访问目标数据,<2>式

表示当前进程也有权限访问目标数据. 如果违反了上述其中一式, 此次操作将会被判定为非法操作, CPU 将会抛出异常, 跳转到一个约定好的代码位置, 然后通知操作系统进行处理.

对 RPL 的补充

你可能会觉得 RPL 十分令人费解, 我们先举一个生活上的例子.

- ❖ 假设你到银行找工作人员办理取款业务, 这时你就相当于 `requestor`, 你的账户相当于 `data`, 工作人员相当于 `current_process`. 业务办理成功是因为
 - 你有权访问自己的账户 (`data.DPL >= requestor.RPL`)
 - 工作人员也有权限对你的账户进行操作 (`data.DPL >= current_process.CPL`)
- ❖ 如果你想从别人的账户中取钱, 虽然工作人员有权访问别人的账户 (`data.DPL >= current_process.CPL`), 但是你却无权访问 (`data.DPL < requestor.RPL`), 因此业务办理失败
- ❖ 如果你打算亲自操作银行系统来取款, 虽然账户是你的 (`data.DPL >= requestor.RPL`), 但是你却无权直接对你的账户金额进行操作 (`data.DPL < current_process.CPL`), 因此你很有可能会被保安抓起来

在计算机中也存在类似的情况: 用户进程 (`requestor`) 想对它自己拥有的数据 (`data`) 进行一些它没有权限的操作, 它就要请求有权限的进程 (`current_process`, 通常是操作系统) 来帮它完成这个操作, 于是就会出现"操作系统代表用户进程进行操作"的场景. 但在真正进行操作之前, 也要检查这些数据是不是真的是用户进程有权使用的数据.

通常情况下, 操作系统运行在 `ring 0`, `CPL` 为 0, 因此有权限访问所有的数据; 而用户进程运行在 `ring 3`, `CPL` 为 3, 这就决定了它只能访问同样处在 `ring 3` 的数据. 这样, 只要操作系统将其私有数据放在 `ring 0` 中, 恶意程序就永远没有办法访问到它们. 这些保护相关的概念和检查过程都是通过硬件实现的, 只要软件运行在硬件上面, 都无法逃出这一天网. 硬件保护机制使得恶意程序永远无法全身而退, 为构建计算机和谐社会作出了巨大的贡献.

这是多美妙的功能! 遗憾的是, 上面提到的很多概念其实只是一带而过, 真正的保护机制也还需要考虑更多的细节. `i386` 手册中专门有一章来描述保护机制, 就已经看出来这并不是简单说说而已. 根据 KISS 法则, 我们并不打算在 `NEMU` 中加入保护机制. 我们让所有用户进程都运行在 `ring 0`, 虽然所有用户进程都有权限执行所有指令, 不过由于 `PA` 中的用户程序都是我们自己编写的, 一切还是在我们的控制范围之内. 毕竟, 我们也已经从上面的故事中体会到保护机制的本质了: 在硬件中加入一些与特权级检查相关的门电路, 如果发现了非法操作, 就会抛出一个异常, 让 CPU 跳转到一个固定的地方, 并进行后续处理.

操作系统的义务

既然操作系统位于 `ring 0` 享受着至高无上的权利, 自然地它也需要履行相应的义务, 那就是: 管理系统中的所有资源, 为用户进程提供相应的服务. 举一个银行的例子, 如果银行连最基本的取款业务都不能办理, 是没有客户愿意光顾它的. 但同时银行也不能允许客户亲自到金库里取款, 而是需要客户按照规定的手续来办理取款业务. 同样地, 操作系统并不允许用户进程直接操作显示器硬件进行输出, 否则恶意程序就很容易往显示器中写入恶意数据, 让屏幕保持黑屏, 影响其它进程的使用. 因此, 用户进程想输出一句话, 也要经过一定的合法手续向操作系统进行申请, 这一合法手续就是系统调用.

我们到银行办理业务的时候, 需要告诉工作人员要办理什么业务, 账号是什么, 交易金额是多少, 这无非是希望工作人员知道我们具体想做什么. 用户进程执行系统调用的时候也是类似的情况, 要通过一种方法描述自己的需求, 然后告诉操作系统. 用来描述需求最方便的手段就是使用通用寄存器了, 用户进程将系统调用的参数依次放入各个寄存器中 (第一个参数放在 `%eax` 中, 第二个参数放在 `%ebx` 中...) 为了让操作系统注意到用户进程提交的申请, 系统调用通常会触发一个异常, 然后陷入操作系统. 在 `GNU/Linux` 中, 系统调用产生的异常通过 `int $0x80` 指令触发. 这个异常和上文提到的非法操作产生的异常不同, 操作系统能够识别它是由系统调用产生的.

`Navy-apps` 已经为用户程序准备好了系统调用的接口了. `navy-apps/libs/libos/src/nanos.c` 中定义的 `_syscall_()` 函数已经蕴含着上述过程:

```
int _syscall_(int type, uintptr_t a0, uintptr_t a1, uintptr_t a2) {
    int ret;
    asm volatile("int $0x80": "=a"(ret): "a"(type), "b"(a0), "c"(a1), "d"(a2));
    return ret;
}
```

上述内联汇编会先把系统调用的参数依次放入 `%eax`, `%ebx`, `%ecx`, `%edx` 四个寄存器中, 然后执行 `int $0x80` 手动触

发一个特殊的异常。操作系统捕获这个异常之后,发现是一个系统调用,就会调出相应的处理函数进行处理,处理结束后设置好返回值,然后返回到上述的内联汇编中。内联汇编最后从`%eax`寄存器中取出系统调用的返回值,并返回给调用该接口的函数,告知其系统调用执行的情况(如是否成功等)。

我们可以在 GNU/Linux 下编写一个程序,来手工触发一次 write 系统调用:

```
const char str[] = "Hello world!\n";

int main() {
    asm volatile ("movl $4, %eax;"           // system call ID, 4 = SYS_write
                  "movl $1, %ebx;"          // file descriptor, 1 = stdout
                  "movl $str, %ecx;"        // buffer address
                  "movl $13, %edx;"         // length
                  "int $0x80");

    return 0;
}
```

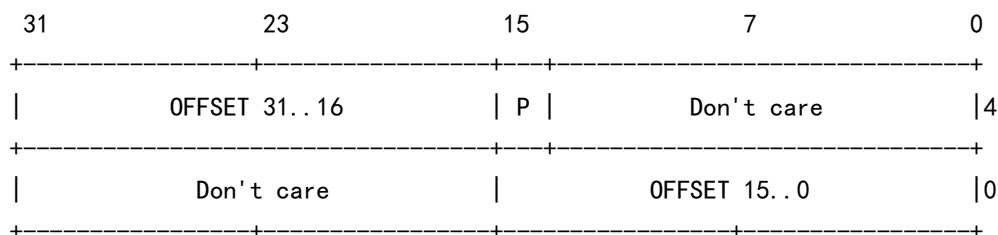
如果你在 64 位操作系统上运行它,你需要在编译的时候加入 `-m32` 参数来生成 32 位的代码. 用户进程执行上述代码,就相当于告诉操作系统:帮我把从 `str` 开始的 13 字节写到 1 号文件中去. 其中“写到 1 号文件中去”的功能相当于输出到屏幕上.

虽然操作系统需要为用户进程服务,但这并不意味着操作系统需要把所有信息都暴露给用户程序.有些信息是用户进程没有必要知道的,也永远不应该知道,例如一些与内存管理相关的数据结构.如果一个恶意程序获得了这些信息,可能会为恶意攻击提供了信息基础.因此,通常不存在一个系统调用来获取这些操作系统的私有数据.

穿越时空的旅程

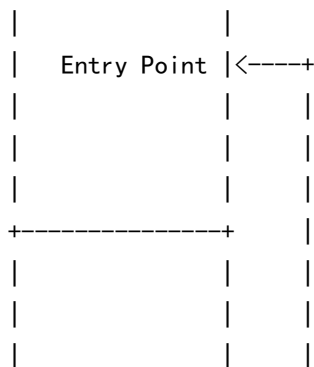
异常是指 CPU 在执行过程中检测到的不正常事件,例如除数为零,无效指令,权限不足等。i386 还向软件提供 int 指令,让软件可以手动产生异常,因此前面提到的系统调用也算是一种特殊的异常。那触发异常之后都发生了些什么呢?我们先来对这一场神秘的时空之旅作一些简单的描述。

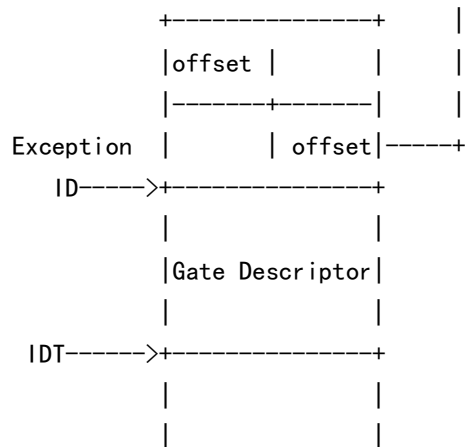
我们之前提到, CPU 检测到异常之后, 就会跳转到一个地方, 这个过程是由位于硬件层次 i386 中断机制支撑的。在 i386 中, 上述跳转的目标通过门描述符(Gate Descriptor)来指示。门描述符是一个 8 字节的结构体, 里面包含着不少细节的信息, 我们在 NEMU 中简化了门描述符的结构, 只保留存在位 P 和偏移量 OFFSET:



P 位用来表示这一个门描述符是否有效, OFFSET 用来指示跳转目标。

为了方便管理门描述符, i386 把内存中的某一段数据专门解释成一个数组 IDT (Interrupt Descriptor Table, 中断描述符表), 数组的一个元素就是一个门描述符。为了从数组中找到一个门描述符, 我们还需要一个索引。对于 CPU 异常来说, 这个索引由 CPU 内部产生 (例如除零异常为 0 号异常), 或者由 int 指令给出 (例如 int \$0x80)。最后, 为了在内存中找到 IDT, i386 使用 IDTR 寄存器来存放 IDT 的首地址和长度。我们需要通过软件代码事先把 IDT 准备好, 然后通过一条特殊的指令 lidt 在 IDTR 中设置好 IDT 的首地址和长度, 这一中断处理机制就可以正常工作了。现在是万事俱备, 等到异常的东风一刮, CPU 就会按照设定好的 IDT 跳转到目标地址:





但感觉有什么不太对劲?异常处理结束之后,我们要怎么返回异常之前的状态呢?为了方便叙述,我们称触发异常之前状态为 **S**. 为了以后能够完美地恢复到 **S**, 在开始真正的处理异常之前应该先把 **S** 保存起来, 等到异常处理结束之后, 才能根据之前保存的信息把计算机恢复到 **S** 的样子. 哪些内容表征了 **S**? 首先当然是 **EIP** 了, 它指示了 **S** 正在执行的指令(或者下一条指令); 然后就是 **EFLAGS** (各种标志位) 和 **CS** (代码段寄存器, 里面包含 **CPL** 的信息). 由于一些特殊的原因, 这三个寄存器的内容必须由硬件来保存. 要将这些信息保存到哪里去呢? 一个合适的地方就是进程的堆栈. 触发异常时, 硬件会自动将 **EFLAGS**, **CS**, **EIP** 三个寄存器的值保存到堆栈上. 于是, 触发异常后硬件的处理如下:

- ❖ 依次将 **EFLAGS**, **CS**, **EIP** 寄存器的值压入堆栈
- ❖ 从 **IDTR** 中读出 **IDT** 的首地址
- ❖ 根据异常(中断)号在 **IDT** 中进行索引, 找到一个门描述符
- ❖ 将门描述符中的 **offset** 域组合成目标地址
- ❖ 跳转到目标地址

需要注意的是, 这些工作都是硬件自动完成的, 不需要程序员编写指令来完成相应的内容. 事实上, 这只是一个简化后的过程, 在真实的计算机上还要处理很多细节问题, 在这里我们就不深究了. **i386** 手册中还记录了处理器对中断号和异常号的分配情况, 并列出了各种异常的详细解释, 需要了解的时候可以进行查阅.

在计算机和谐社会中, 大部分门描述符都不能让用户进程随意使用, 否则恶意程序就可以通过 **int** 指令欺骗操作系统. 例如恶意程序执行 **int \$0x2** 来谎报电源掉电, 扰乱其它进程的正常运行. 因此执行 **int** 指令也需要进行特权级检查, 但 **PA** 中就不实现这一保护机制了, 具体的检查规则我们也就不展开讨论了, 需要了解的时候请查阅 **i386** 手册.

加入 ASYE

在 **AM** 的模型中, 异常处理的能力被划分到 **ASYE** 模块中. 老规矩, 我们还是分别从 **NEMU** 和 **AM** 两个角度来体会硬件和软件如何相互协助来支持 **ASYE** 的功能.

准备 IDT

首先是要准备一个有意义的 **IDT**, 这样以后触发异常时才能跳转到正确的目标地址. 具体的, 你需要在 **NEMU** 中添加 **IDTR** 寄存器和 **lidt** 指令. 然后在 **nanos-lite/src/main.c** 中定义宏 **HAS_ASYE**, 这样以后, **Nanos-lite** 会多进行一项初始化工作: 调用 **init_irq()** 函数, 这最终会调用位于 **nexus-am/am/arch/x86-nemu/src/asye.c** 中的 **_asye_init()** 函数. **_asye_init()** 函数会做两件事情, 第一件就是初始化 **IDT**:

- ❖ 代码定义了一个结构体数组 **idt**, 它的每一项是一个门描述符结构体
- ❖ 在相应的数组元素中填写有意义的门描述符, 例如编号为 **0x80** 的门描述符就是将来系统调用的入口地址. 需要注意的是, 框架代码中还是填写了完整的门描述符(包括上文中提到的 **don't care** 的域), 这主要是为了在 **QEMU** 中进行 **differential testing** 时也能跳转到正确的入口地址. **QEMU** 实现了完整的中断机制, 如果只填写简化版的门描述符, 就无法在 **QEMU** 中正确运行. 但我们无需了解其中的细节, 只需要知道代码已经填写了正确的门描述符即可.

- ❖ 在 **IDTR** 中设置 **idt** 的首地址和长度

_asye_init() 函数做的第二件事是注册一个事件处理函数, 这个事件处理函数由 **_asyn_init()** 的调用者提供. 关于事件处理函数, 我们会在下文进行更多的介绍.

触发异常

为了测试是否已经成功准备 **IDT**, 我们还需要真正触发一次异常, 看是否正确地跳转到目标地址. 具体的, 你

需要在 NEMU 中实现 `raise_intr()` 函数(在 `nemu/src/cpu/intr.c` 中定义)来模拟上文提到的 i386 中断机制的处理过程:

```
void raise_intr(uint8_t NO, vaddr_t save_addr) {
    /* TODO: Trigger an interrupt/exception with ``NO``.
     * That is, use ``NO`` to index the IDT.
     */
}
```

需要注意的是:

- ❖ PA 不涉及特权级的切换, 查阅 i386 手册的时候你不需要关心和特权级切换相关的内容.
- ❖ 通过 IDTR 中的地址对 IDT 进行索引的时候, 需要使用 `vaddr_read()`.
- ❖ PA 中不实现分段机制, 没有 CS 寄存器的概念. 但为了在 QEMU 中顺利进行 differential testing, 我们还是在 `cpu` 结构体中添加一个 CS 寄存器, 并在 `restart()` 函数中将其初始化为 8.
- ❖ 由于中断机制需要对 EFLAGS 进行压栈, 为了配合 differential testing, 我们还需要在 `restart()` 函数中将 EFLAGS 初始化为 0x2.
- ❖ 执行 `int` 指令后保存的 EIP 指向的是 `int` 指令的下一条指令, 这有点像函数调用, 具体细节可以查阅 i386 手册.
- ❖ 你需要在 `int` 指令的 `helper` 函数中调用 `raise_intr()`, 而不要把中断机制的代码放在 `int` 指令的 `helper` 函数中实现, 因为在后面我们会再次用到 `raise_intr()` 函数.

实现中断机制

你需要实现上文提到的 `lidt` 指令和 `int` 指令, 并实现 `raise_intr()` 函数. 实现正确后, 重新在 Nanos-lite 上运行 dummy 程序, 如果你看到在 `vecsys()` (在 `nexus-am/am/arch/x86-nemu/src/trap.S` 中定义)附近触发了未实现指令, 说明你的中断机制实现正确.

保存现场

成功跳转到入口函数 `vecsys()` 之后, 我们就要在软件上开始真正的异常处理过程了. 但是, 进行异常处理的时候不可避免地需要用到通用寄存器, 然而看看现在的通用寄存器, 里面存放的都是异常触发之前的内容. 这些内容也是现场的一部分, 如果不保存就覆盖它们, 将来就无法恢复异常触发之前的状态了. 但硬件并不负责保存它们, 因此需要通过软件代码来保存它们的值. i386 提供了 `pusha` 指令, 用于把通用寄存器的值压入堆栈.

`vecsys()` 会压入错误码和异常号 `#irq`, 然后跳转到 `asm_trap()`. 需要注意的是, `push imm8` 指令需要对立即数进行符号扩展, 这一点在 i386 手册中并没有明确说明. 在 `asm_trap()` 中, 代码将会把用户进程的通用寄存器保存到堆栈上. 这些寄存器的内容连同之前保存的错误码, `#irq`, 以及硬件保存的 EFLAGS, CS, EIP, 形成了 `trap frame` (陷阱帧)的数据结构. 我们知道栈帧记录了函数调用时的状态, 而相应地, 陷阱帧则完整记录了用户进程触发异常时现场的状态, 将来恢复现场就靠它了.

对比异常与函数调用

我们知道进行函数调用的时候也需要保存调用者的状态: 返回地址, 以及调用约定 (calling convention) 中需要调用者保存的寄存器. 而进行异常处理之前却要保存更多的信息. 尝试对比它们, 并思考两者保存信息不同是什么原因造成的.

注意到 `trap frame` 是在堆栈上构造的. 接下来代码将会把当前的 `%esp` 压栈, 并调用 C 函数 `irq_handle()` (在 `nexus-am/am/arch/x86-nemu/src/asye.c` 中定义).

诡异的代码

`trap.S` 中有一行 `pushl %esp` 的代码, 乍看之下其行为十分诡异. 你能结合前后的代码理解它的行为吗? Hint: 不用想太多, 其实都是你学过的知识.

重新组织 TrapFrame 结构体

你的任务如下:

- ❖ 实现 `pusha` 指令, 你需要注意压栈的顺序, 更多信息请查阅 i386 手册.
- ❖ 理解 `trap frame` 形成的过程, 然后重新组织 `nexus-am/am/arch/x86-nemu/include/arch.h` 中定义的 `_RegSet` 结构体的成员, 使得这些成员声明的顺序和 `nexus-am/am/arch/x86-nemu/src/trap.S` 中构造的 `trap frame` 保持一致.

实现正确之后, `irq_handle()` 以及后续代码就可以正确地使用 `trap frame` 了. 重新在 Nanos-lite 上运行 dummy

程序, 你会看到在 `nanos-lite/src/irq.c` 中的 `do_event()` 函数中触发了 `BAD TRAP`:

```
[src/irq.c, 5, do_event] {kernel} system panic: Unhandled event ID = 8
```

事件分发

`irq_handle()` 的代码会把异常封装成事件, 然后调用在 `_asye_init()` 中注册的事件处理函数, 将事件交给它来处理. 在 `Nanos-lite` 中, 这一事件处理函数是 `nanos-lite/src/irq.c` 中的 `do_event()` 函数. `do_event()` 函数会根据事件类型再次进行分发. 我们刚才触发了一个未处理的 8 号事件, 这其实是一个系统调用事件 `_EVENT_SYSCALL` (在 `nexus-am/am/am.h` 中定义). 在识别出系统调用事件后, 需要调用 `do_syscall()` (在 `nanos-lite/src/syscall.c` 中定义) 进行处理.

系统调用处理

我们终于正式进入系统调用的处理函数中了. `do_syscall()` 首先通过宏 `SYSCALL_ARG1()` 从现场 `r` 中获取用户进程之前设置好的系统调用参数, 通过第一个参数 - 系统调用号 - 进行分发. 但目前 `Nanos-lite` 没有实现任何系统调用, 因此触发了 `panic`.

添加一个系统调用比你想象中要简单, 所有信息都已经准备好了. 我们只需要在分发的过程中添加相应的系统调用号, 并编写相应的系统调用处理函数 `sys_xxx()`, 然后调用它即可. 回过头来看 `dummy` 程序, 它触发了一个号码为 0 的 `SYS_none` 系统调用. 我们约定, 这个系统调用什么都不用做, 直接返回 1.

处理系统调用的最后一件事就是设置系统调用的返回值. 我们约定系统调用的返回值存放在系统调用号所在的寄存器中, 所以我们只需要通过 `SYSCALL_ARG1()` 来进行设置就可以了.

恢复现场

系统调用处理结束后, 代码将会一路返回到 `trap.S` 的 `asm_trap()` 中. 接下来的事情就是恢复用户进程的现场. `asm_trap()` 将根据之前保存的 `trap frame` 中的内容, 恢复用户进程的通用寄存器 (注意 `trap frame` 中的 `%eax` 已经被设置成系统调用的返回值了), 并直接弹出一些不再需要的信息, 最后执行 `iret` 指令. `iret` 指令用于从异常处理代码中返回, 它将栈顶的三个元素来依次解释成 `EIP`, `CS`, `EFLAGS`, 并恢复它们. 用户进程可以通过 `%eax` 寄存器获得系统调用的返回值, 进而得知系统调用执行的结果. 在它看来, 这次时空之旅就好像没有发生过一样.

实现系统调用

你需要:

- ❖ 在 `do_event()` 中识别出系统调用事件 `_EVENT_SYSCALL`, 然后调用 `do_syscall()`.
- ❖ 在 `nexus-am/am/arch/x86-nemu/include/arch.h` 中实现正确的 `SYSCALL_ARGx()` 宏, 让它们从作为参数的现场 `reg` 中获得正确的系统调用参数寄存器.
- ❖ 添加 `SYS_none` 系统调用.
- ❖ 设置系统调用的返回值.
- ❖ 实现 `popa` 和 `iret` 指令.

重新运行 `dummy` 程序, 如果你的实现正确, 你会看到 `dummy` 程序又触发了一个号码为 4 的系统调用. 查看 `nanos-lite/src/syscall.h`, 你会发现它是一个 `SYS_exit` 系统调用. 这说明之前的 `SYS_none` 已经成功返回, 触发 `SYS_exit` 是因为 `dummy` 已经执行完毕, 准备退出了.

你需要实现 `SYS_exit` 系统调用, 它会接收一个退出状态的参数, 用这个参数调用 `_halt()` 即可. 实现成功后, 再次运行 `dummy` 程序, 你会看到 `GOOD TRAP` 的信息.

需要提醒的是, `ASYE` 还有其它的 API, 但我们暂时不会用到, 现在可以先忽略它们.

温馨提示

PA3 阶段 1 到此结束.

在操作系统上运行 Hello World

成功运行 `dummy` 程序后, 我们已经把系统调用的整个流程都摸清楚了.

标准输出

`Navy-apps` 中提供了一个 `hello` 测试程序 (`navy-apps/tests/hello`), 它首先通过 `write()` 来输出一句话, 然后通过 `printf()` 来不断输出. 为了运行它, 我们只需要再实现 `SYS_write` 系统调用即可. 根据 `write` 的函数声明 (参考 `man 2 write`), 在 `do_syscall()` 中识别出系统调用号是 `SYS_write` 之后, 检查 `fd` 的值, 如果 `fd` 是 1 或 2 (分

别代表 `stdout` 和 `stderr`), 则将 `buf` 为首地址的 `len` 字节输出到串口 (使用 `_putc()` 即可). 最后还要设置正确的返回值, 否则系统调用的调用者会认为 `write` 没有成功执行, 从而进行重试. 至于 `write` 系统调用的返回值是什么, 请查阅 `man 2 write`. 另外不要忘记在 `navy-apps/libs/libos/src/nanos.c` 的 `_write()` 中调用系统调用接口函数.

事实上, 我们平时使用的 `printf()`, `cout` 这些库函数和库类, 对字符串进行格式化之后, 最终也是通过系统调用进行输出. 这些都是 "系统调用封装成库函数" 的例子. 系统调用本身对操作系统的各种资源进行了抽象, 但为了给上层的程序员提供更好的接口 (`beautiful interface`), 库函数会再次对部分系统调用再次进行抽象. 例如 `fwrite()` 这个库函数用于往文件中写入数据, 在 GNU/Linux 中, 它封装了 `write()` 系统调用. 另一方面, 系统调用依赖于具体的操作系统, 因此库函数的封装也提高了程序的可移植性: 在 Windows 中, `fwrite()` 封装了 `WriteFile()` 系统调用, 如果在代码中直接使用 `WriteFile()` 系统调用, 把代码放到 GNU/Linux 下编译就会产生链接错误.

并不是所有的库函数都封装了系统调用, 例如 `strcpy()` 这类字符串处理函数就不需要使用系统调用. 从某种程度上来说, 库函数的抽象确实方便了程序员, 使得他们不必关心系统调用的细节.

实现 `SYS_write` 系统调用之后, 我们已经为 "使用 `printf()`" 扫除了最大的障碍了, 因为 `printf()` 进行字符串格式化之后, 最终会通过 `write()` 系统调用进行输出. 这些工作, `Navy-apps` 中的 `newlib` 库已经为我们准备好了.

在 Nanos-lite 上运行 Hello world

实现 `write()` 系统调用, 然后把 `Nanos-lite` 上运行的用户程序切换到 `hello` 程序并运行:

- ❖ 切换到 `navy-apps/tests/hello/` 目录下执行 `make` 编译 `hello` 程序
- ❖ 修改 `nanos-lite/Makefile` 中 `ramdisk` 的生成规则, 把 `ramdisk` 中的唯一的文件换成 `hello` 程序:

```
--- nanos-lite/Makefile
+++ nanos-lite/Makefile
@@ -9,2 +9,2 @@
OBJCOPY_FLAG = -S --set-section-flags .bss=alloc,contents -O binary
-OBJS_COPY_FILE = $(NAVY_HOME)/tests/dummy/build/dummy-x86
+OBJS_COPY_FILE = $(NAVY_HOME)/tests/hello/build/hello-x86
```

- ❖ 在 `nanos-lite/Makefile` 下执行 `make update` 更新 `ramdisk`
- ❖ 重新编译 `Nanos-lite` 并运行

堆区管理

如果你在 `Nanos-lite` 中的 `sys_write()` 中通过 `Log()` 观察 `write` 系统调用的调用情况, 你会发现用户程序通过 `printf()` 输出的时候是逐个字符地调用 `write` 来输出的. 事实上, 用户程序在第一次调用 `printf()` 的时候会尝试通过 `malloc()` 申请一片缓冲区, 来存放格式化的内容. 若申请失败, 就会逐个字符进行输出.

`malloc()/free()` 库函数的作用是在用户程序的堆区中申请/释放一块内存区域. 堆区的使用情况是由 `libc` 来进行管理的, 但堆区的大小却需要通过系统调用向操作系统提出更改. 这是因为, 堆区的本质是一片内存区域, 当需要调整堆区大小的时候, 实际上是在调整用户程序可用的内存区域. 事实上, 一个用户程序可用的内存区域要经过操作系统的分配和管理的. 想象一下, 如果一个恶意程序可以不经操作系统的同意, 就随意使用其它程序的内存区域, 将会引起灾难性的后果. 当然, 目前 `Nanos-lite` 只是个单任务操作系统, 不存在多个程序的概念. 在 `PA4` 中, 你将会对这个问题有更深刻的认识.

调整堆区大小是通过 `sbrk()` 库函数来实现的, 它的原型是 `void* sbrk(intptr_t increment)` 用于将用户程序的 `program break` 增长 `increment` 字节, 其中 `increment` 可为负数. 所谓 `program break`, 就是用户程序的数据段 (`data segment`) 结束的位置. 我们知道可执行文件里面有代码段和数据段, 链接的时候 `ld` 会默认添加一个名为 `_end` 的符号, 来指示程序的数据段结束的位置. 用户程序开始运行的时候, `program break` 会位于 `_end` 所指示的位置, 意味着此时堆区的大小为 0. `malloc()` 被第一次调用的时候, 会通过 `sbrk(0)` 来查询用户程序当前 `program break` 的位置, 之后就可以通过后续的 `sbrk()` 调用来动态调整用户程序 `program break` 的位置了. 当前 `program break` 和和其初始值之间的区间就可以作为用户程序的堆区, 由 `malloc()/free()` 进行管理. 注意用户程序不应该直接使用 `sbrk()`, 否则将会扰乱 `malloc()/free()` 对堆区的管理记录.

在 `Navy-apps` 的 `Newlib` 中, `sbrk()` 最终会调用 `_sbrk()`, 它在 `navy-apps/libs/libos/src/nanos.c` 中定义. 框架代码让 `_sbrk()` 总是返回 -1, 表示堆区调整失败, 于是 `printf()` 会认为无法在堆区中申请用于格式化的缓冲区, 只好逐个字符地输出. 但如果堆区总是不可用, `Newlib` 中很多库函数的功能将无法使用, 因此现在你需要实现 `_sbrk()` 了. 为了实现 `_sbrk()` 的功能, 我们还需要提供一个用于设置堆区大小的系统调用. 在 GNU/Linux 中, 这个系统调用是 `SYS_brk`, 它接收一个参数 `addr`, 用于指示新的 `program break` 的位置. `_sbrk()` 通过记录的方式来对用户程

序的 program break 位置进行管理,其工作方式如下:

- ❖ program break 一开始的位置位于_end
- ❖ 被调用时,根据记录的 program break 位置和参数 increment,计算出新 program break
- ❖ 通过 SYS_brk 系统调用来让操作系统设置新 program break
- ❖ 若 SYS_brk 系统调用成功,该系统调用会返回 0,此时更新之前记录的 program break 的位置,并将旧 program break 的位置作为_sbrk()的返回值返回
- ❖ 若该系统调用失败,_sbrk()会返回-1

上述代码是在用户层的库函数中实现的,我们还需要在 Nanos-lite 中实现 SYS_brk 的功能.目前 Nanos-lite 还是一个单任务操作系统,空闲的内存都可以让用户程序自由使用,因此我们只需要让 SYS_brk 系统调用总是返回 0 即可,表示堆区大小的调整总是成功.

实现堆区管理

根据上述内容在 Nanos-lite 中实现 SYS_brk 系统调用,然后在用户层实现_sbrk().你可以通过 man 2 sbrk 来查阅 libc 中 brk()和 sbrk()的行为,另外通过 man 3 end 来查阅如何使用_end 符号.

需要注意的是,调试的时候不要在_sbrk()中通过 printf()进行输出,这是因为 printf()还是会尝试通过 malloc()来申请缓冲区,最终会再次调用_sbrk(),造成死递归.你可以通过 sprintf()先把调试信息输出到一个字符串缓冲区中,然后通过 write 系统调用进行输出.

如果你的实现正确,你将会在 Nanos-lite 中看到 printf()将格式化完毕的字符串通过一次 write 系统调用进行输出,而不是逐个字符地进行输出.

缓冲区与系统调用开销

你已经了解系统调用的过程了.事实上,如果通过系统调用千辛万苦地陷入操作系统只是为了输出区区一个字符,那就太不划算了.于是有了 batching 的技术:将一些简单的任务累积起来,然后再一次性进行处理.缓冲区是 batching 技术的核心,libc 中的输入输出函数正是通过缓冲区来将输入输出累积起来,然后再通过一次系统调用进行处理.例如通过一个 1024 字节的缓冲区,就可以通过一次系统调用直接输出 1024 个字符,而不需要通过 1024 次系统调用来逐个字符地输出.显然,后者的开销比前者大得多.

有兴趣的同学可以在 GNU/Linux 上编写相应的程序,来粗略测试一下一次 write 系统调用的开销,然后和[这篇文章](#)对比一下.

简易文件系统

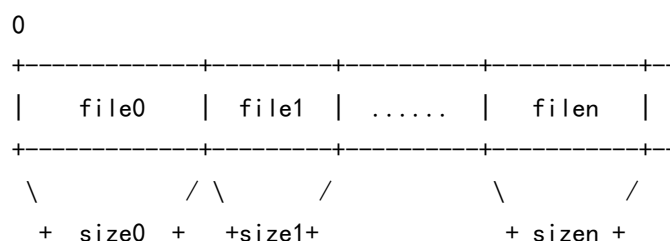
我们的 ramdisk 已经提供了读写接口,使得我们可以很方便地访问某一个位置的数据.目前 ramdisk 中只有一个文件,使用起来没什么繁琐的地方.但如果文件的数量增加之后,我们就要知道哪个文件在 ramdisk 的什么位置.这对 Nanos-lite 来说貌似没什么困难的地方,但对用户程序来说,它怎么知道文件位于 ramdisk 的哪一个位置呢?更何况文件会动态地增删,用户程序并不知情.这说明,把 ramdisk 的读写接口直接提供给用户程序来使用是不可行的.操作系统还需要在存储介质的驱动程序之上为用户程序提供一种更高级的抽象,那就是文件.

文件的本质就是字节序列,另外还由一些额外的属性构成.在这里,我们先讨论普通意义上的文件.这样,那些额外的属性就维护了文件到 ramdisk 存储位置的映射.为了管理这些映射,同时向上层提供文件操作的接口,我们需要在 Nanos-lite 中实现一个文件系统.

不要被"文件系统"四个字吓到了,我们对文件系统的需求并不是那么复杂:

- ❖ 每个文件的大小是固定的
- ❖ 写文件时不允许超过原有文件的大小
- ❖ 文件的数量是固定的,不能创建新文件
- ❖ 没有目录

既然文件的数量和大小都是固定的,我们自然可以把每一个文件分别固定在 ramdisk 中的某一个位置.这些简化的特性大大降低了文件系统的实现难度.当然,真实的文件系统远远比这个简易文件系统复杂.我们约定文件从 ramdisk 的最开始一个挨着一个地存放:



为了记录 ramdisk 中各个文件的名称和大小, 我们还需要一张"文件记录表". Nanos-lite 的 Makefile 已经提供了维护这些信息的脚本, 先对 nanos-lite/Makefile 作如下修改:

```
--- nanos-lite/Makefile
+++ nanos-lite/Makefile
@@ -34,2 +34,2 @@
-update: update-ramdisk-objcopy src/syscall.h
+update: update-ramdisk-fsimg src/syscall.h
@touch src/initrd.S
```

然后运行 `make update` 就会自动编译 Navy-apps 里面的所有程序, 并把 navy-apps/fsimg/ 目录下的所有内容整合成 ramdisk 镜像, 同时生成这个 ramdisk 镜像的文件记录表 nanos-lite/src/files.h. 需要注意的是, 并不是 Navy-apps 里面的所有程序都能在 Nanos-lite 上运行, 有些程序需要更多系统调用的支持才能运行, 例如 NWM 和 NTerm, 我们并不打算在 PA 中运行这些程序. "文件记录表" 其实是一个数组, 数组的每个元素都是一个结构体:

```
typedef struct {
    char *name;           // 文件名
    size_t size;          // 文件大小
    off_t disk_offset;    // 文件在 ramdisk 中的偏移
} Finfo;
```

在我们的简易文件系统里面, 这三项信息都是固定不变的. 其中的文件名和我们平常使用的习惯不太一样: 由于我们的简易文件系统中没有目录, 我们把目录分隔符/也认为是文件名的一部分, 例如/bin/hello 是一个完整的文件名. 这种做法其实也隐含了目录的层次结构, 对于文件数量不多的情况, 这种做法既简单又奏效. 有了这些信息, 就已经可以实现最基本的文件读写操作了:

```
ssize_t read(const char *filename, void *buf, size_t len);
ssize_t write(const char *filename, void *buf, size_t len);
```

但在真实的操作系统中, 这种直接用文件名来作为读写操作参数的做法却所有缺陷. 例如, 我们在用 less 工具浏览文件的时候:

```
cat file | less
```

cat 工具希望把文件内容写到 less 工具的标准输入中, 但我们却无法用文件名来标识 less 工具的标准输入! 实际上, 操作系统中确实存在不少"没有名字"的文件. 为了统一管理它们, 我们希望通过一个编号来表示文件, 这个编号就是文件描述符(file descriptor). 一个文件描述符对应一个正在打开的文件, 由操作系统来维护文件描述符到具体文件的映射. 于是我们很自然地通过 open() 系统调用来打开一个文件, 并返回相应的文件描述符

```
int open(const char *pathname, int flags, int mode);
```

在 Nanos-lite 中, 由于简易文件系统里的文件数目是固定的, 我们可以简单地把文件记录表的下标作为相应文件的文件描述符返回给用户程序. 在这以后, 所有文件操作都通过文件描述符来标识文件:

```
ssize_t read(int fd, void *buf, size_t len);
ssize_t write(int fd, const void *buf, size_t len);
int close(int fd);
```

另外, 我们也不希望每次读写操作都需要从头开始. 于是我们需要为每一个已经打开的文件引入偏移量属性 open_offset, 来记录目前文件操作的位置. 每次对文件读写了多少个字节, 偏移量就前进多少.

```
--- nanos-lite/src/fs.c
+++ nanos-lite/src/fs.c
@@ -3,5 +3,6 @@
typedef struct {
    char *name;           // 文件名
    size_t size;          // 文件大小
    off_t disk_offset;    // 文件在 ramdisk 中的偏移
+    off_t open_offset;   // 文件被打开之后的读写指针
} Finfo;
```

事实上在真正的操作系统中, 把偏移量放在文件记录表中维护会导致用户程序无法实现某些功能. 但解释这个问题需要理解一些超出课程范围的知识, 我们在此就不展开叙述了. 而且由于 Nanos-lite 是一个精简版的操作系统, 上述问题暂时不会出现, 为了简化实现, 我们还是把偏移量放在文件记录表中进行维护.

偏移量可以通过 `lseek()` 系统调用来调整:

```
off_t lseek(int fd, off_t offset, int whence);
```

为了方便用户程序进行标准输入输出, 操作系统准备了三个默认的文件描述符:

```
#define FD_STDIN 0
#define FD_STDOUT 1
#define FD_STDERR 2
```

它们分别对应标准输入 `stdin`, 标准输出 `stdout` 和标准错误 `stderr`. 我们经常使用的 `printf`, 最终调用 `write(FD_STDOUT, buf, len)` 进行输出; 而 `scanf` 将会通过调用 `read(FD_STDIN, buf, len)` 进行读入.

`nanos-lite/src/fs.c` 中定义的 `file_table` 会包含 `nanos-lite/src/files.h`, 其中前面还有 6 个特殊的文件, 前三个分别是 `stdin`, `stdout` 和 `stderr` 的占位表项, 它们只是为了保证我们的简易文件系统和约定的标准输入输出的文件描述符保持一致, 例如根据约定 `stdout` 的文件描述符是 1, 而我们添加了三个占位表项之后, 文件记录表中的 1 号下标也就不会分配给其它的普通文件了. 后面三个是特殊的文件, 我们会在后面来介绍它们, 目前可以先忽略它们. 根据以上信息, 我们就可以在文件系统中实现以下的文件操作了:

```
int fs_open(const char *pathname, int flags, int mode);
ssize_t fs_read(int fd, void *buf, size_t len);
ssize_t fs_write(int fd, const void *buf, size_t len);
off_t fs_lseek(int fd, off_t offset, int whence);
int fs_close(int fd);
```

这些文件操作实际上是相应的系统调用在内核中的实现. 你可以通过 `man` 查阅它们的功能, 例如 `man 2 open` 其中 2 表示查阅和系统调用相关的 manual page. 实现这些文件操作的时候注意以下几点:

- ❖ 由于简易文件系统中每一个文件都是固定的, 不会产生新文件, 因此 "`fs_open()` 没有找到 `pathname` 所指示的文件" 属于异常情况, 你需要使用 `assertion` 终止程序运行.
- ❖ 为了简化实现, 我们允许所有用户程序都可以对所有已存在的文件进行读写, 这样以后, 我们在实现 `fs_open()` 的时候就可以忽略 `flags` 和 `mode` 了.
- ❖ 使用 `ramdisk_read()` 和 `ramdisk_write()` 来进行文件的真正读写.
- ❖ 由于文件的大小是固定的, 在实现 `fs_read()`, `fs_write()` 和 `fs_lseek()` 的时候, 注意偏移量不要越过文件的边界.
- ❖ 除了写入 `stdout` 和 `stderr` 之外 (用 `_putc()` 输出到串口), 其余对于 `stdin`, `stdout` 和 `stderr` 这三个特殊文件的操作可以直接忽略.
- ❖ 由于我们的简易文件系统没有维护文件打开的状态, `fs_close()` 可以直接返回 0, 表示总是关闭成功.

最后你还需要在 `Nanos-lite` 和 `Navy-apps` 的 `libos` 中添加相应的系统调用, 来调用相应的文件操作.

让 loader 使用文件

我们之前是让 `loader` 来直接调用 `ramdisk_read()` 来加载用户程序. `ramdisk` 中的文件数量增加之后, 这种方式就不合适了, 我们首先需要让 `loader` 享受到文件系统的便利.

你需要先实现 `fs_open()`, `fs_read()` 和 `fs_close()`, 这样就可以在 `loader` 中使用文件名来指定加载的程序了, 例如 `"/bin/hello"`. 我们还需要让 `fs_read()` 知道文件的大小, 我们可以在文件系统中添加一个辅助函数 `size_t fs_filesz(int fd)`; 它用于返回文件描述符 `fd` 所描述的文件的大小.

实现之后, 以后更换用户程序只需要修改传入 `loader()` 函数的文件名即可, 无需更新 `ramdisk` 的内容 (除非 `ramdisk` 上的内容确实需要更新, 例如重新编译了 `Navy-apps` 的程序).

实现完整的文件系统

实现 `fs_write()` 和 `fs_lseek()`, 然后运行测试程序 `/bin/text`. 这个测试程序用于进行一些简单的文件读写和定位操作. 如果你的实现正确, 你将会看到程序输出 `PASS!!!` 的信息.

温馨提示

PA3 阶段 2 到此结束.

一切皆文件

我们已经提供了完整的文件系统, 用户程序已经可以读写普通的文件了. 想想我们在 AM 上运行的打字游戏, 读入按键/查询时钟/更新屏幕其实也是用户程序的合理需求, 操作系统也需要提供支持. 一种最直接的方式, 就是为每个功能单独提供一个系统调用, 用户程序通过这些系统调用, 就可以直接使用相应的功能了. 然而这种做法却存在不少问题:

- ❖ 首先, 设备的类型五花八门, 其功能更是数不胜数, 要为它们分别实现系统调用来给用户程序提供接口, 本身就已经缺乏可行性了;
- ❖ 此外, 由于设备的功能差别较大, 若提供的接口不能统一, 程序之间的交互就会变得困难.

我们在上一小节中提到, 文件的本质就是字节序列. 事实上, 计算机系统中到处都是字节序列(如果只是无序的字节集合, 计算机要如何处理?), 我们可以轻松地举出很多例子:

- 内存是以字节编址的, 天然就是一个字节序列, 因而我们之前使用的 `ramdisk` 作为字节序列也更加显而易见
- 管道(`shell` 命令中的 `|`) 是一种先进先出的字节序列, 本质上它是内存中的一个队列缓冲
- 磁盘也可以看成一个字节序列: 我们可以为磁盘上的每一个字节进行编号, 例如第 `x` 柱面第 `y` 磁头第 `z` 扇区中的第 `n` 字节, 把磁盘上的所有字节按照编号的大小进行排列, 便得到了一个字节序列
- `socket` (网络套接字) 也是一种字节序列, 它有一个缓冲区, 负责存放接收到的网络数据包, 上层应用将 `socket` 中的内容看做是字节序列, 并通过一些特殊的文件操作来处理它们
- 操作系统的一些信息可以以字节序列的方式暴露给用户, 例如 CPU 的配置信息
- 操作系统提供的一些特殊的功能, 如随机数生成器, 也可以看成一个无穷长的字节序列
- 甚至一些非存储类型的硬件也可以看做是字节序列: 我们在键盘上按顺序敲入按键的编码形成了一个字节序列, 显示器上每一个像素的内容按照其顺序也可以看做是字节序列...

既然文件就是字节序列, 那很自然地, 上面这些五花八门的字节序列应该都可以看成文件. Unix 就是这样做的, 因此有 "一切皆文件" (Everything is a file) 的说法. 这种做法最直观的好处就是为不同的事物提供了统一的接口: 我们可以使用文件的接口来操作计算机上的一切, 而不必对它们进行详细的区分: 例如 `nanos-lite/Makefile` 中通过管道把各个 `shell` 工具的输入输出连起来, 生成文件记录表

```
wc -c $(FSIMG_FILES) | grep -v 'total$$' | sed -e 's+ $(FSIMG_PATH)+ +' |  
awk -v sum=0 '{print "\x7b\x22" $$2 "\x22\x2c " $$1 "\x2c " sum "\x7d\x2c";sum += $
```

以十六进制的方式查看磁盘上的内容

```
head -c 512 /dev/sda | hd
```

查看 CPU 的配置信息

```
cat /proc/cpuinfo | vim -
```

而

```
#include "/dev/urandom"
```

则会将 `urandom` 中的内容包含到源文件中: 由于 `urandom` 是一个长度无穷的字节序列, 提交一个包含上述内容的程序源文件将会令一些检测功能不强的 `Online Judge` 平台直接崩溃.

"一切皆文件" 的抽象使得我们可以通过标准工具很容易完成一些在 Windows 下不易完成的工作, 这其实体现了 Unix 哲学的部分内容: 每个程序采用文本文件作为输入输出, 这样可以使程序之间易于合作. GNU/Linux 继承自 Unix, 也自然继承了这种优秀的特性. 为了向用户程序提供统一的抽象, `Nanos-lite` 也尝试将 IOE 抽象成文件.

首先当然是来看输出设备. 串口已经被抽象成 `stdout` 和 `stderr` 了, 我们无需担心. 至于 VGA, 程序为了更新屏幕, 只需要将像素信息写入 VGA 的显存即可. 于是, `Nanos-lite` 需要做的, 便是把显存抽象成文件. 显存本身也是一段存储空间, 它以行优先的方式存储了将要在屏幕上显示的像素. `Nanos-lite` 和 `Navy-apps` 约定, 把显存抽象成文件 `/dev/fb` (`fb` 为 `frame buffer` 之意), 它需要支持写操作和 `lseek`, 以便于用户程序把像素更新到屏幕的指定位置上.

除此之外, 用户程序还需要获得屏幕大小的信息, 然后才能决定如何更好地显示像素内容. `Nanos-lite` 和 `Navy-apps` 约定, 屏幕大小的信息通过 `/proc/dispinfo` 文件来获得, 它需要支持读操作. `/proc/dispinfo` 内容的

一个例子如下:

```
WIDTH:640
HEIGHT:480
```

需要注意的是, `/dev/fb` 和 `/proc/dispinfo` 都是特殊的文件, 文件记录表中有它们的文件名, 但它们的实体并不在 `ramdisk` 中. 因此, 我们需要在 `fs_read()` 和 `fs_write()` 的实现中对它们进行"重定向", 以 `fs_write()` 为例:

```
ssize_t fs_write(int fd const void *buf, size_t len) {
    // ...
    switch (fd) {
        case FD_STDOUT:
        case FD_STDERR:
            // call _putc()
            break;

        case FD_FB:
            // write to frame buffer
            break;

        default:
            // write to ramdisk
            break;
    }
}
```

把 VGA 显存抽象成文件

你需要在 `Nanos-lite` 中

- 在 `init_fs()` (在 `nanos-lite/src/fs.c` 中定义) 中对文件记录表中 `/dev/fb` 的大小进行初始化, 你需要使用 IOE 定义的 API 来获取屏幕的大小.
- 实现 `fb_write()` (在 `nanos-lite/src/device.c` 中定义), 用于把 `buf` 中的 `len` 字节写到屏幕上 `offset` 处. 你需要先从 `offset` 计算出屏幕上的坐标, 然后调用 IOE 的 `_draw_rect()` 接口.
- 在 `init_device()` (在 `nanos-lite/src/device.c` 中定义) 中将 `/proc/dispinfo` 的内容提前写入到字符串 `dispinfo` 中. 实际的屏幕大小信息已经记录在 AM 的 IOE 接口中, 你需要在 `Nanos-lite` 中获取它们.
- 实现 `dispinfo_read()` (在 `nanos-lite/src/device.c` 中定义), 用于把字符串 `dispinfo` 中 `offset` 开始的 `len` 字节写到 `buf` 中.
- 在文件系统中添加对 `/dev/fb` 和 `/proc/dispinfo` 这两个特殊文件的支持.

让 `Nanos-lite` 加载 `/bin/bmptest`, 如果实现正确, 你将会看到屏幕上显示 ProjectN 的 Logo.

最后我们来看输入设备. 输入设备有键盘和时钟, 我们需要把它们输入包装成事件. 一种简单的方式是把事件以文本的形式表现出来, 我们定义以下事件, 一个事件以换行符 `\n` 结束:

- `t 1234`: 返回系统启动后的时间, 单位为毫秒;
- `kd RETURN/ku A`: 按下/松开按键, 按键名称全部大写, 使用 AM 中定义的按键名

我们采用文本形式来描述事件有两个好处, 首先文本显然是一种字节序列, 这使得事件很容易抽象成文件; 此外文本方式使得用户程序可以容易可读地解析事件的内容. `Nanos-lite` 和 `Navy-apps` 约定, 上述事件抽象成文件 `/dev/events`, 它需要支持读操作, 用户程序可以从其中一次读出一个输入事件. 需要注意的是, 由于时钟事件可以任意时刻进行读取, 我们需要优先处理按键事件, 当不存在按键事件的时候, 才返回时钟事件, 否则用户程序将永远无法读到按键事件.

把设备输入抽象成文件

你需要在 `Nanos-lite` 中

- ❖ 实现 `events_read()` (在 `nanos-lite/src/device.c` 中定义), 把事件写入到 `buf` 中, 最长写入 `len` 字节, 然后返回写入的实际长度. 其中按键名已经在字符串数组 `names` 中定义好了. 你需要借助 IOE 的 API 来获得设备的输入.
- ❖ 在文件系统中添加对 `/dev/events` 的支持.

让 `Nanos-lite` 加载 `/bin/events`, 如果实现正确, 你会看到程序输出时间事件的信息, 敲击按键时会输出按键事件的信息.

运行仙剑奇侠传

原版的仙剑奇侠传是针对 Windows 平台开发的, 因此它并不能在 GNU/Linux 中运行(你知道为什么吗?), 也不能在 NEMU 中运行. 网友 weimingzhi 开发了一款基于 SDL 库, 跨平台的仙剑奇侠传, 工程叫 SDLPAL. 你可以通过 git clone 命令把 SDLPAL 克隆到本地, 然后把仙剑奇侠传的数据文件(我们已经把数据文件上传到提交网站上)放在工程目录下, 执行 make 编译 SDLPAL, 编译成功后就可以玩了. 更多的信息请参考 SDLPAL 工程中的 README 说明.

我们的框架代码已经把 SDLPAL 移植到 Navy-apps 中了. 移植的主要工作就是把应用层之下提供给仙剑奇侠传的所有 API 重新实现一遍, 因为这些 API 大多都依赖于操作系统提供的运行时环境, 我们需要根据 Navy-apps 提供的运行时环境重写它们. 主要包括以下三部分内容:

- C 标准库
- 浮点数
- SDL 库

Navy-apps 中的 newlib 已经提供了 C 标准库的功能, 我们无需额外移植. 关于浮点数的移植工作, 我们会在 PA5 中再来讨论, 目前先忽略它. 为了移植 SDL 库相关的代码, Navy-apps 把时钟, 键盘, 显示的功能封装成 NDL (NJU DirectMedia Layer) 多媒体库, 其中封装了我们之前实现的/dev/fb 和/dev/events 的读写. 为了用 NDL 的 API 来替代原来 SDL 的相应功能, 移植工作需要对 SDLPAL 进行了少量修改, 包括去掉了声音, 修改了和按键相关的处理, 把我们关心的与 NDL 相关的功能整理到 hal/hal.c 中, 一些我们不必关心的实现则整理到 unused/目录下. 框架代码已经把这些移植工作都做好了, 目前你不需要编写额外的代码来进行移植.

在 NEMU 中运行仙剑奇侠传

终于到了激动人心的时刻了! 我们已经通过文件的抽象向仙剑奇侠传提供了所有它需要的功能了. 从提交网站上下载仙剑奇侠传的数据文件, 并放到 navy-apps/fsimg/share/games/pal/目录下, 更新 ramdisk 之后, 在 Nanos-lite 中加载并运行/bin/pal.

在我们提供的文件数据中包含一些游戏存档, 可以读取迷宫中的存档, 与怪物进行战斗. 但战斗需要进行一些浮点数相关的计算, 而 NEMU 目前没有实现浮点数, 因而不能成功进行战斗. 我们会在 PA5 中再来解决浮点数的问题, 目前我们先暂时不触发战斗, 可以先通过"新的故事"进行游戏.



必答题

文件读写的具体过程 仙剑奇侠传中有以下行为:

- ❖ 在 navy-apps/apps/pal/src/global/global.c 的 PAL_LoadGame() 中通过 fread() 读取游戏存档
- ❖ 在 navy-apps/apps/pal/src/hal/hal.c 的 redraw() 中通过 NDL_DrawRect() 更新屏幕

请结合代码解释仙剑奇侠传, 库函数, libos, Nanos-lite, AM, NEMU 是如何相互协助, 来分别完成游戏存档的读取和屏幕的更新.

温馨提示

PA3 到此结束.